

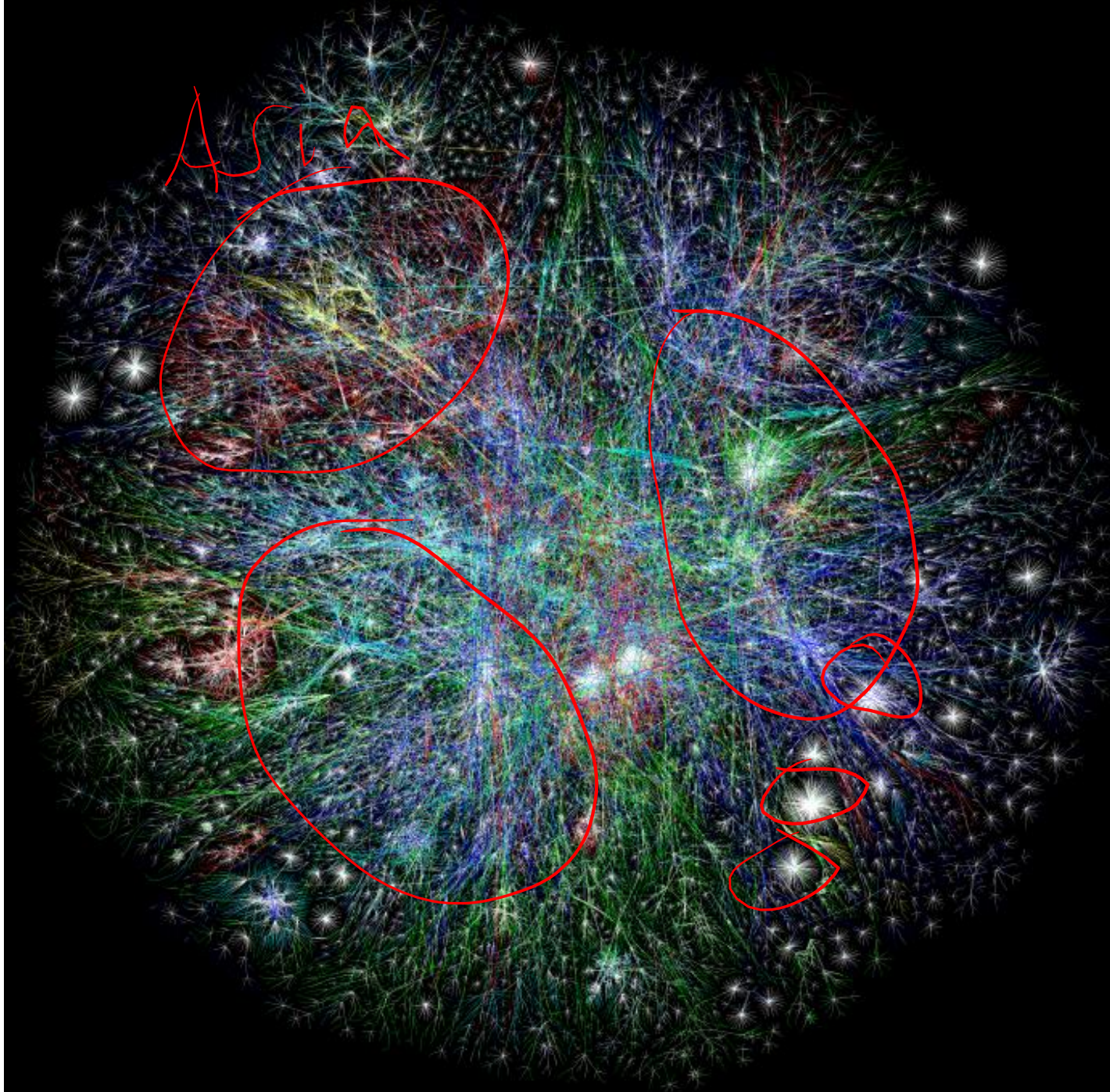


# CS 225

## Data Structures

**Volodymyr Kindratenko**

*Slides courtesy of Wade Fagen-Ulmschneider*

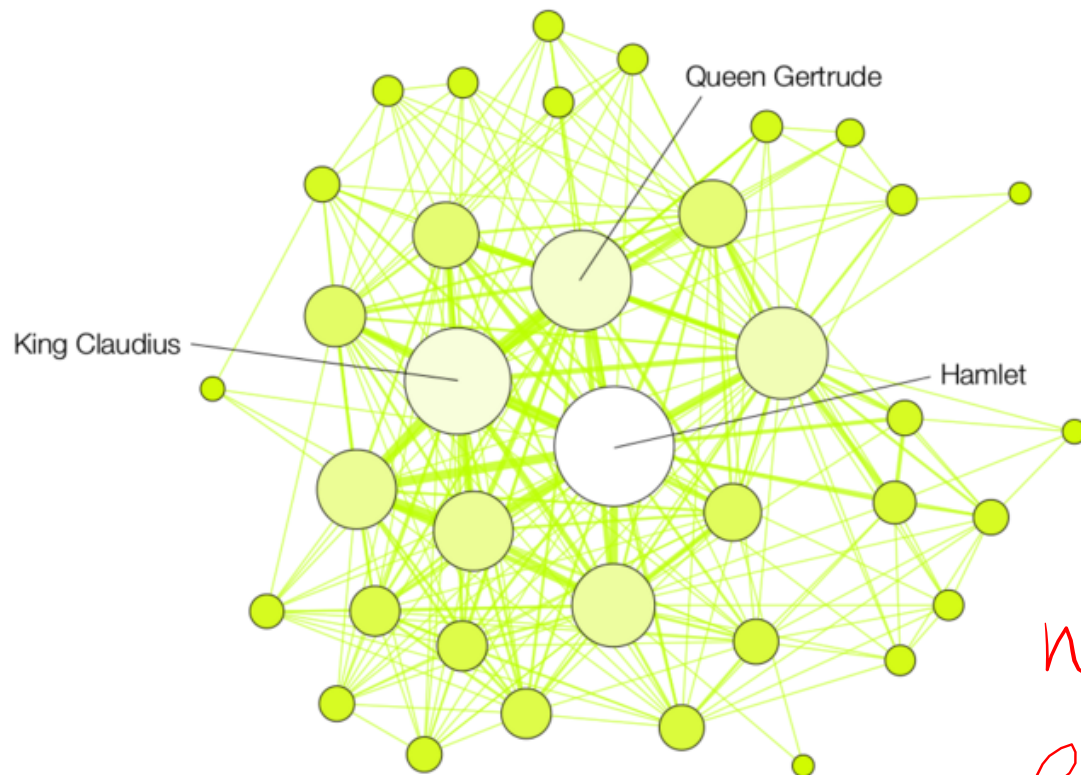


node  
router  
edge  
Connect  
B/T router

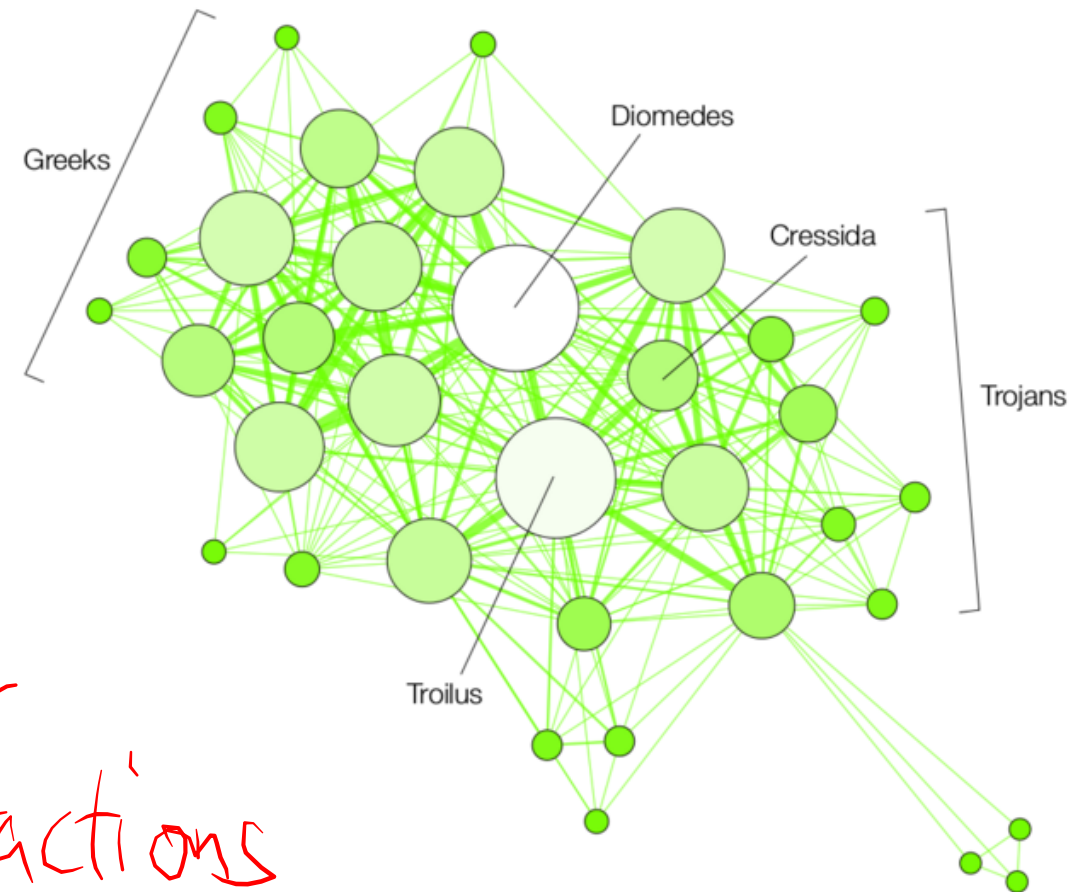
## The Internet 2003

*The OPTE Project (2003)*

Map of the entire internet; nodes  
are routers; edges are connections.



HAMLET



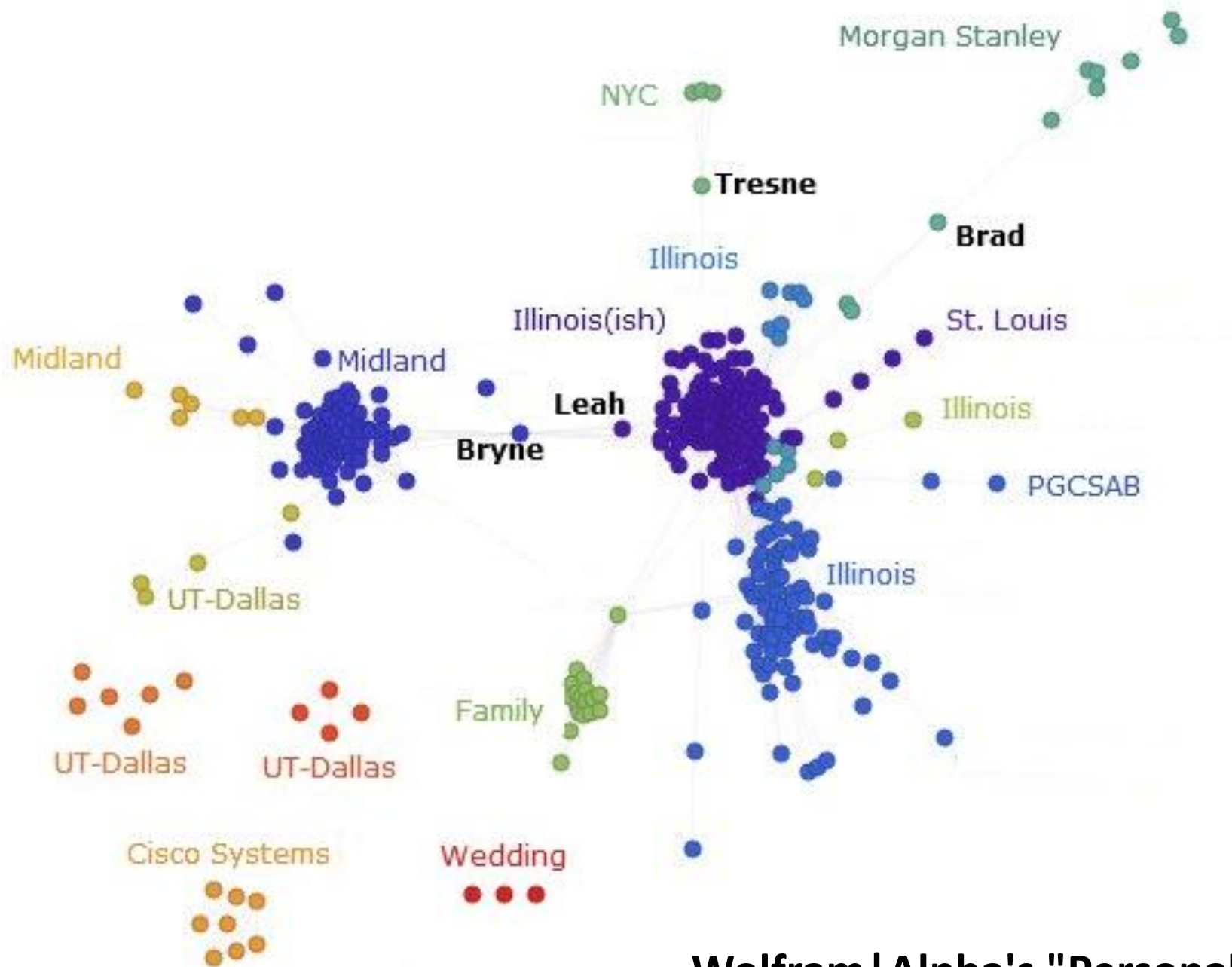
TROILUS AND CRESSIDA

n: char  
e: interactions  
w: # inter

**Who's the real main character in Shakespearean tragedies?**

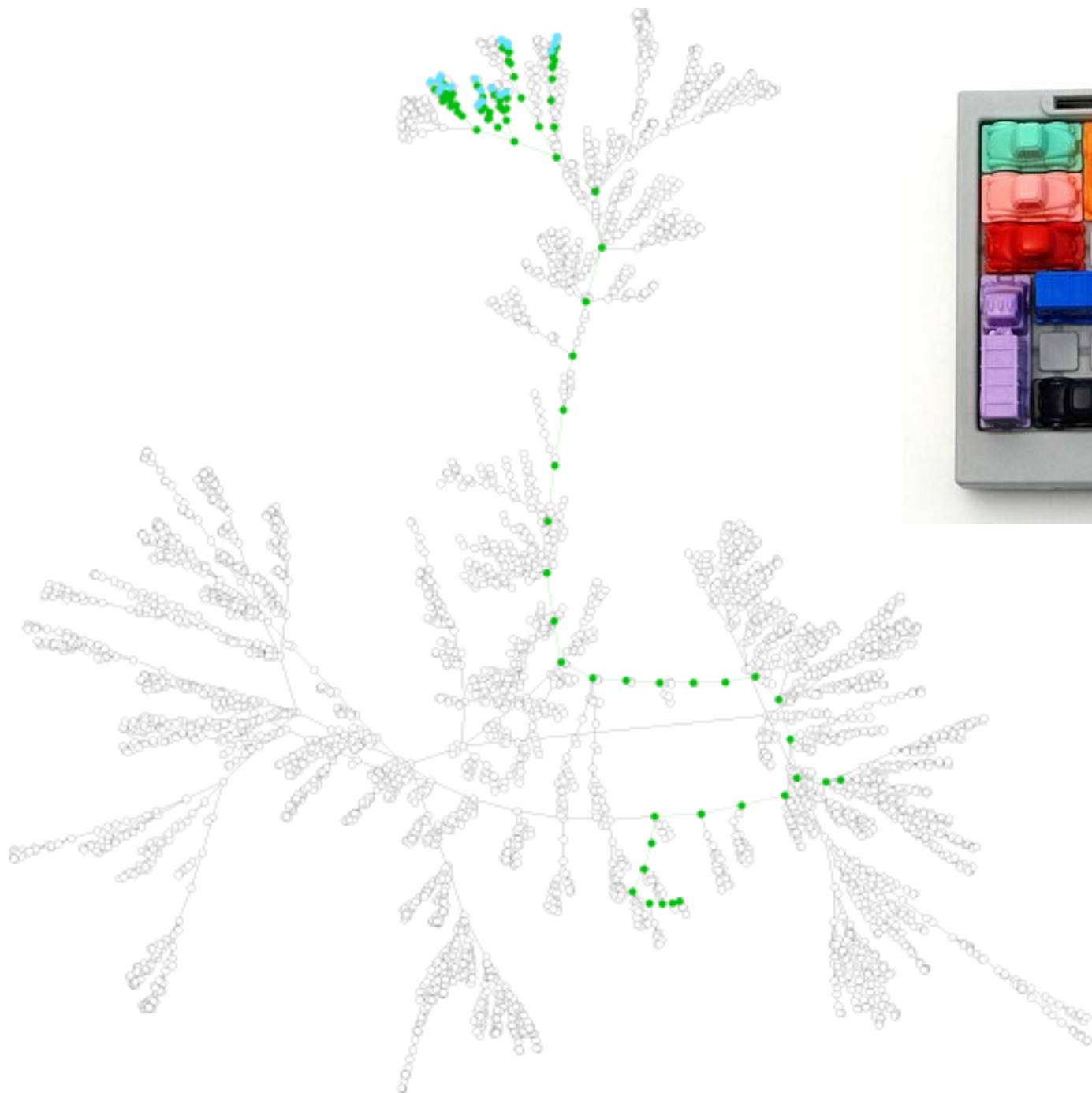
*Martin Grandjean (2016)*

<https://www.pbs.org/newshour/arts/whos-the-real-main-character-in-shakespearean-tragedies-heres-what-the-data-say>



**Wolfram|Alpha's "Personal Analytics" for Facebook**

*Generated: April 2013 using Wade Fagen-Ulmschneider's Profile Data*

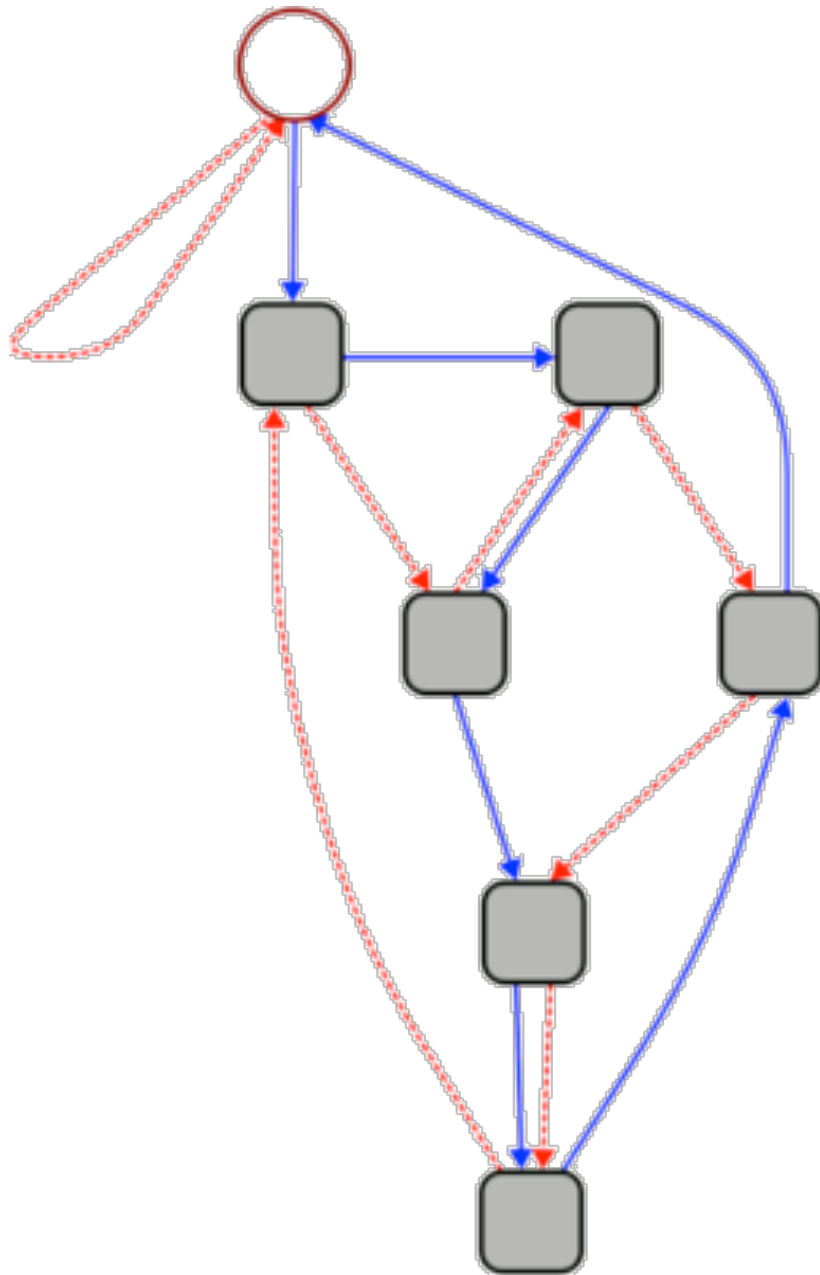


$n$  : game state  
 $e$  : move

## **“Rush Hour” Solution**

*Unknown Source*

*Presented by Cinda Heeren, 2016*



This graph can be used to quickly calculate whether a given number is divisible by 7.

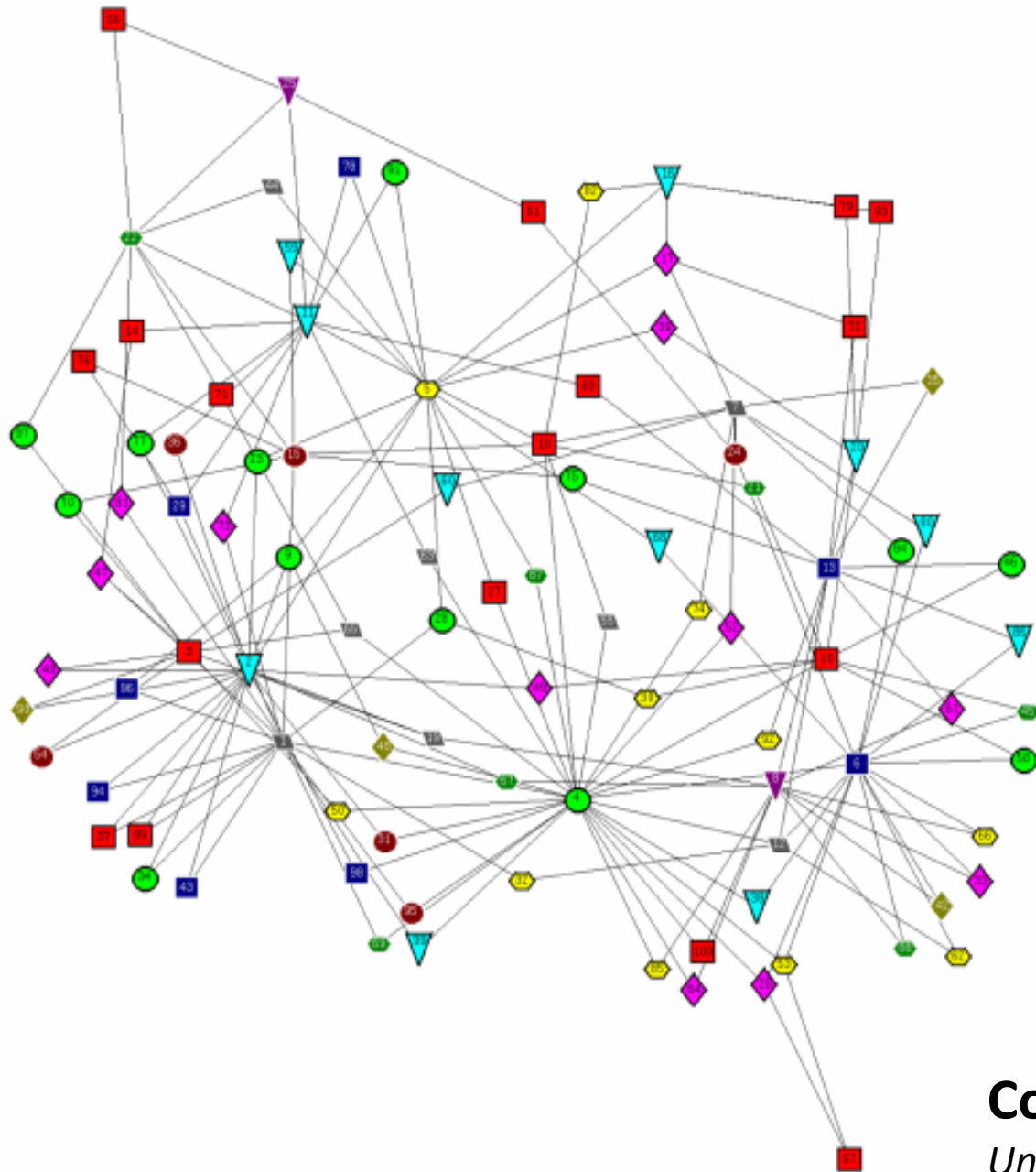
1. Start at the circle node at the top.
2. For each digit **d** in the given number, follow **d** blue (solid) edges in succession. As you move from one digit to the next, follow **1** red (dashed) edge.
3. If you end up back at the circle node, your number is divisible by 7.

3703

**“Rule of 7”**

*Unknown Source*

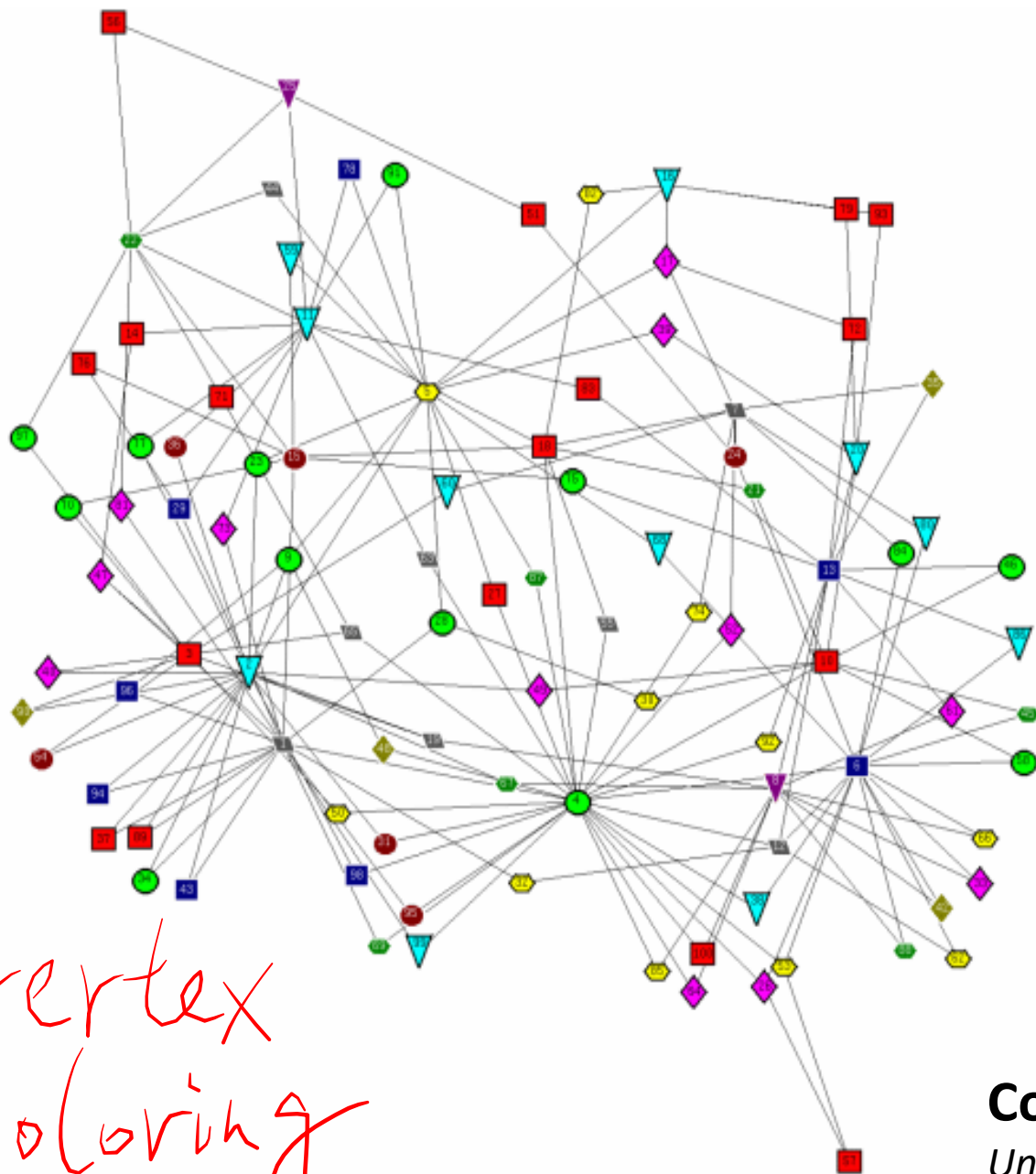
*Presented by Cinda Heeren, 2016*



## Conflict-Free Final Exam Scheduling Graph

*Unknown Source*

*Presented by Cinda Heeren, 2016*



vertex  
coloring

$n$ : course  
 $(u, v)$ : edge

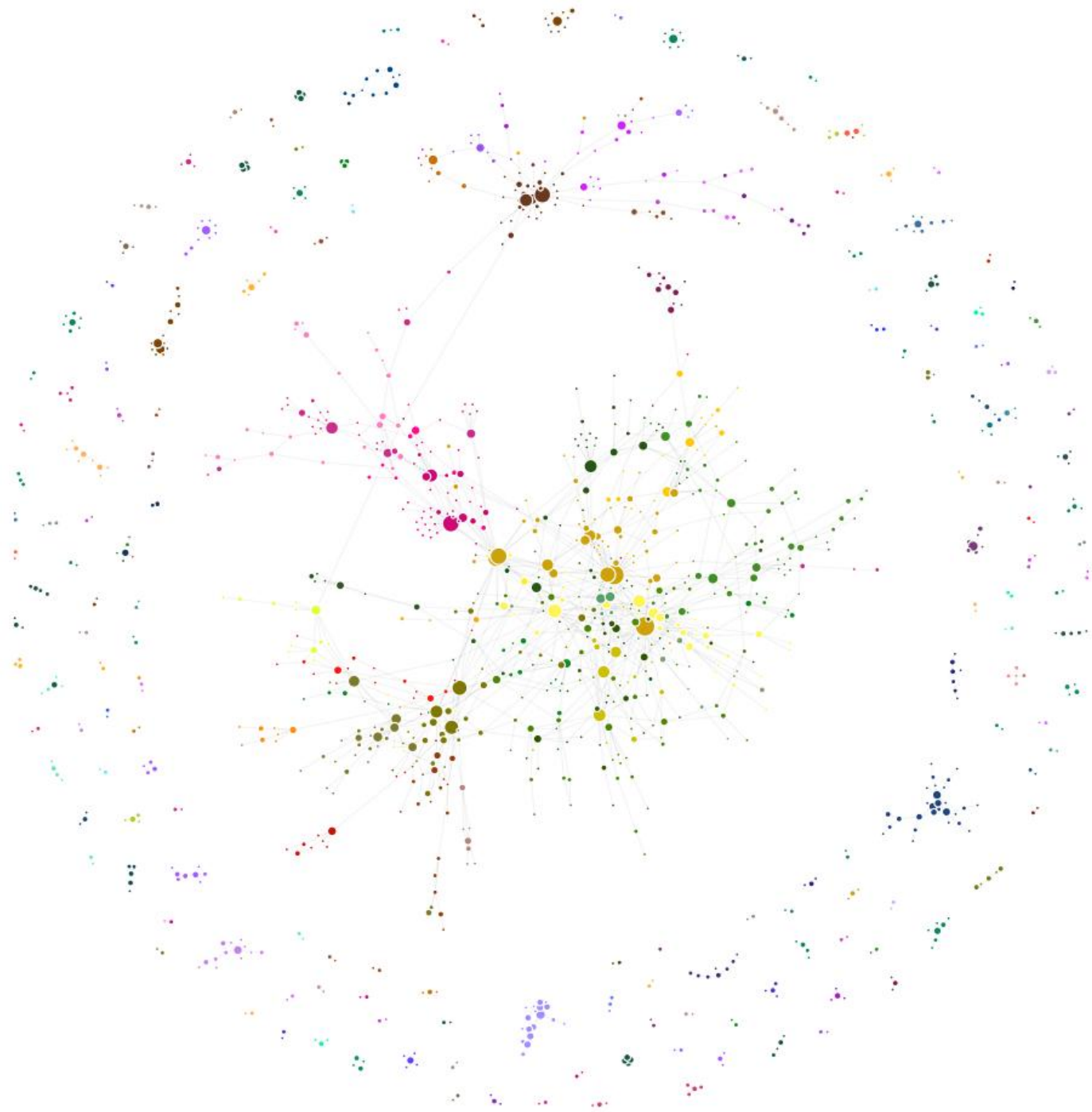
---

If a student enrolled  
in both courses

**Conflict-Free Final Exam Scheduling Graph**

*Unknown Source*

*Presented by Cinda Heeren, 2016*

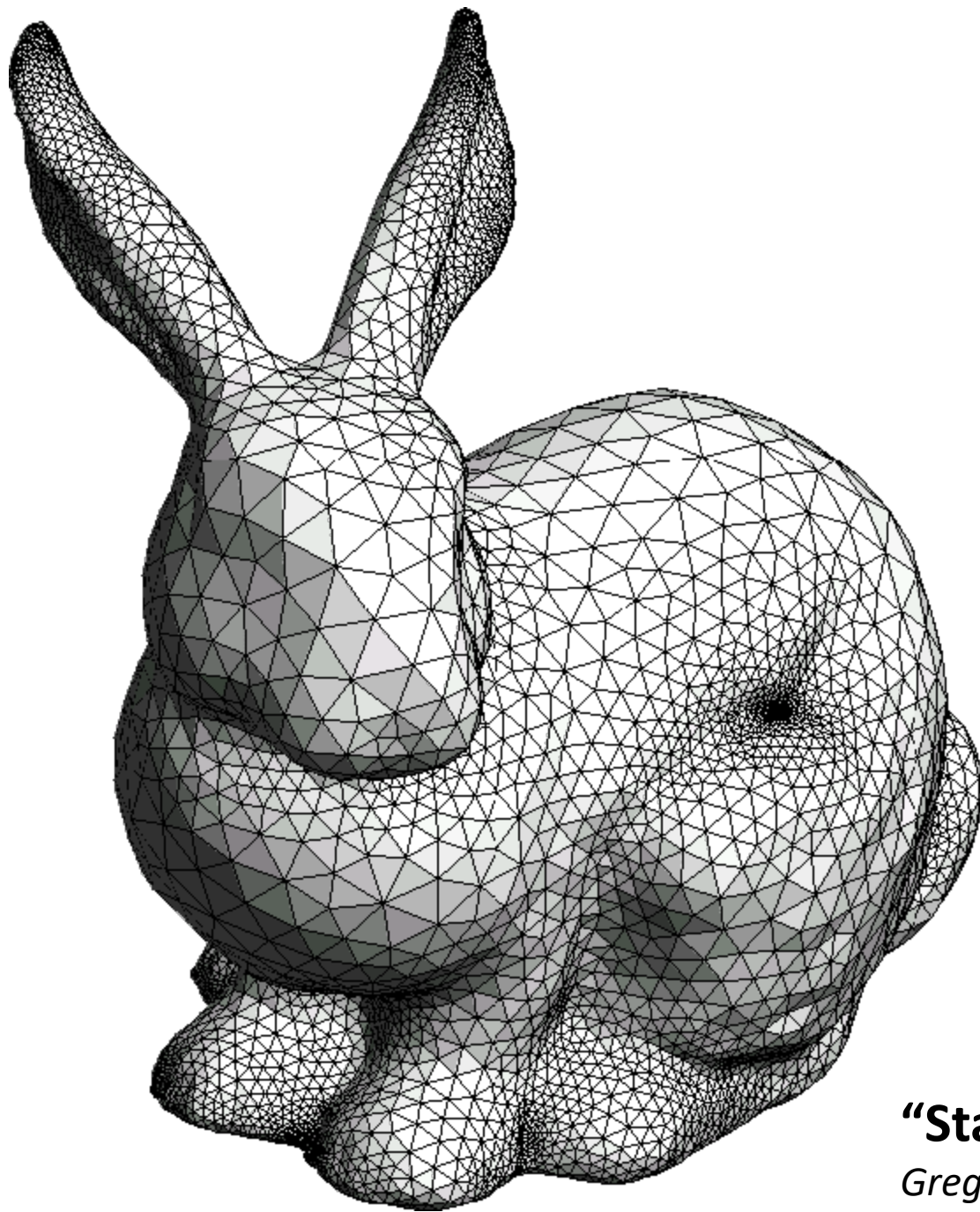


## Class Hierarchy At University of Illinois Urbana-Champaign

*A. Mori, W. Fagen-Ulmschneider, C. Heeren*

Graph of every course at UIUC; nodes are courses, edges are prerequisites

[http://waf.cs.illinois.edu/discovery/class\\_hierarchy\\_at\\_illinois/](http://waf.cs.illinois.edu/discovery/class_hierarchy_at_illinois/)

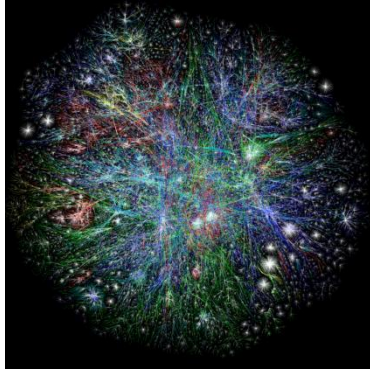


$n$  : vertex  
 $e$  : triangle side

**“Stanford Bunny”**

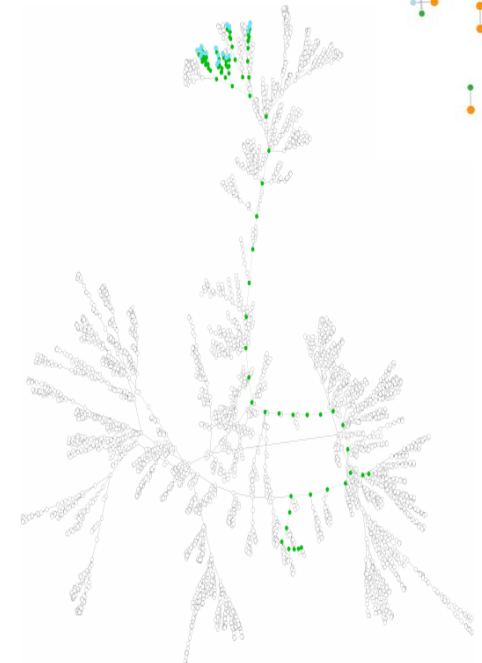
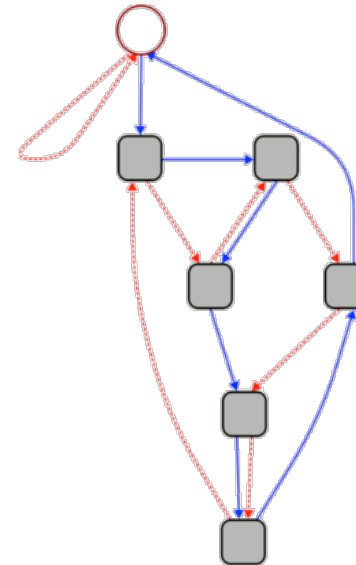
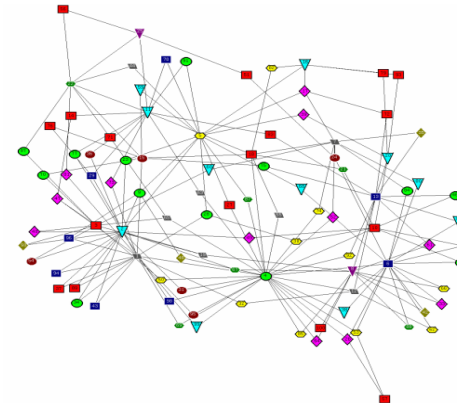
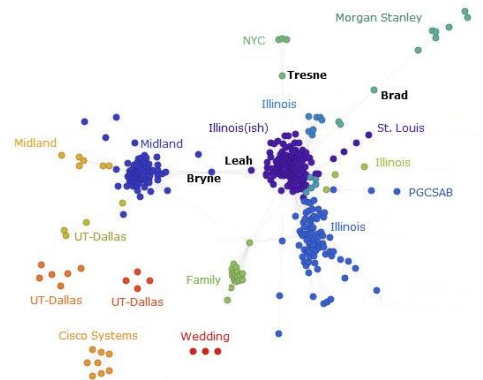
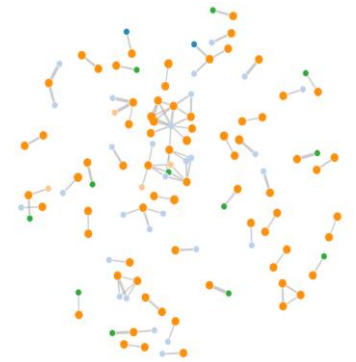
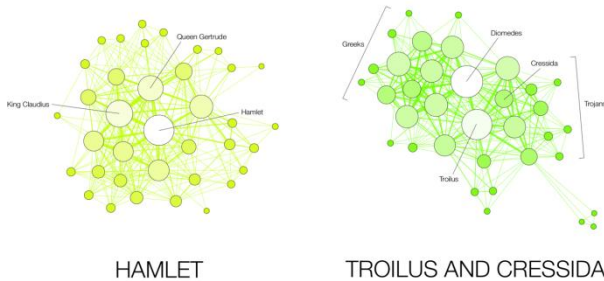
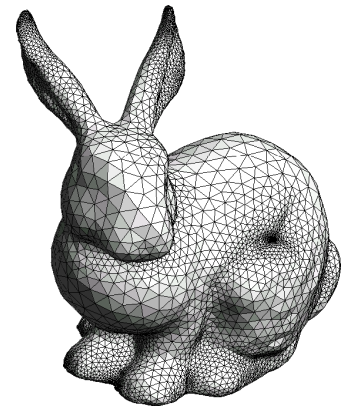
*Greg Turk and Mark Levoy (1994)*

# Graphs



**To study all of these structures:**

1. A common vocabulary
2. Graph implementations
3. Graph traversals
4. Graph algorithms

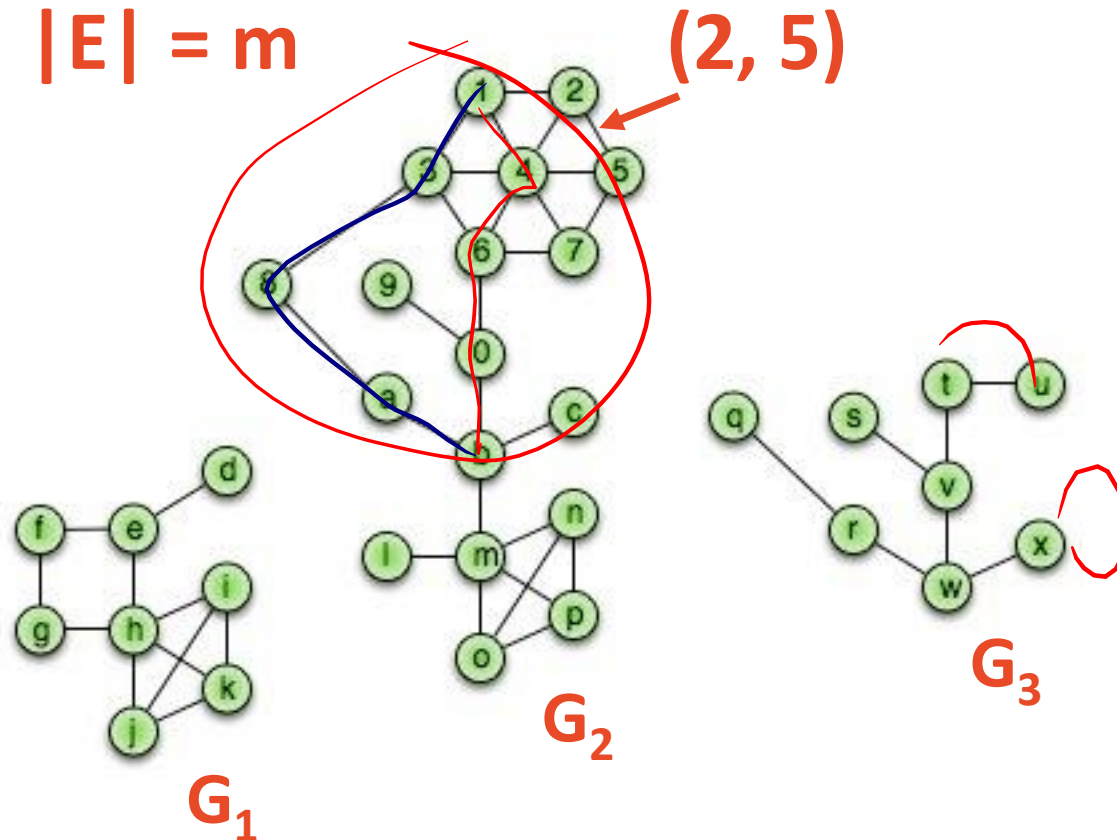


# Graph Vocabulary

$$G = (V, E)$$

$$|V| = n$$

$$|E| = m$$



Incident Edges:

$$I(v) = \{ (x, v) \text{ in } E \}$$

Degree(v):  $||$

Adjacent Vertices:

$$A(v) = \{ x : (x, v) \text{ in } E \}$$

Path( $G_2$ ): Sequence of vertices connected by edges

Cycle( $G_1$ ): Path with a common begin and end vertex.

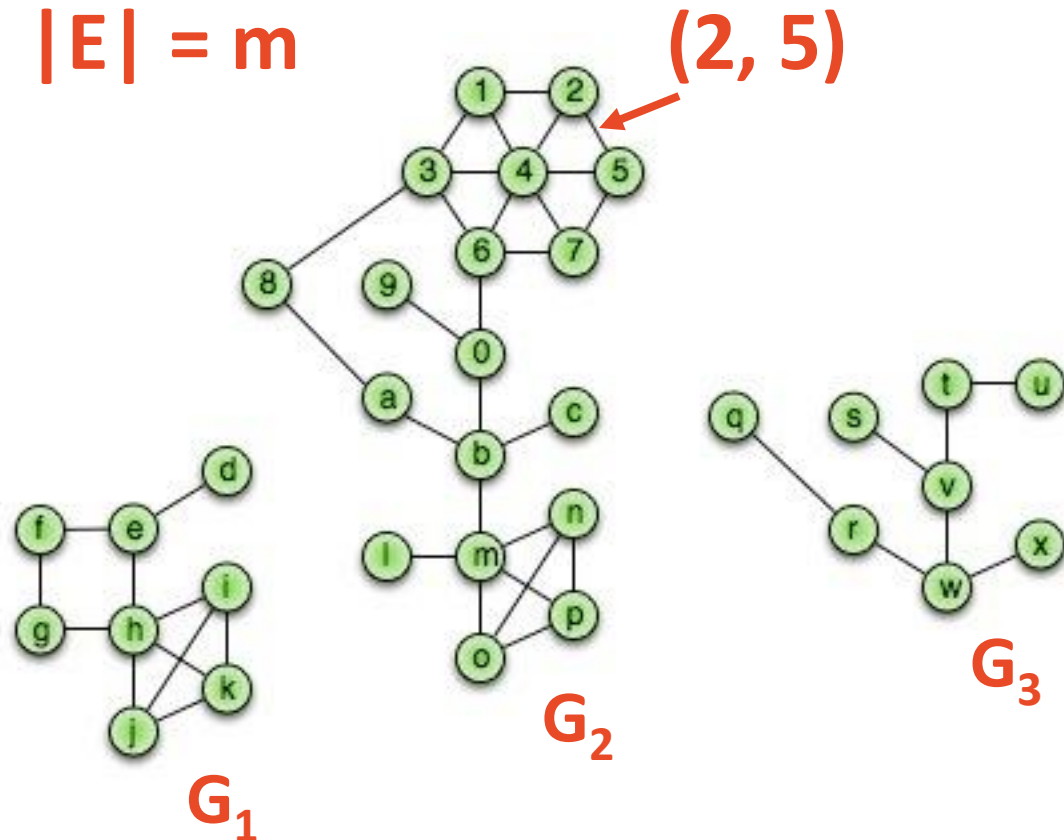
Simple Graph( $G$ ): A graph with no self loops or multi-edges.

# Graph Vocabulary

$$G = (V, E)$$

$$|V| = n$$

$$|E| = m$$



Subgraph( $G$ ):

$$G' = (V', E'):$$

$V' \subseteq V, E' \subseteq E$ , and

$$(u, v) \in E' \rightarrow u \in V', v \in V'$$

Complete subgraph( $G$ )

Connected subgraph( $G$ )

Connected component( $G$ )

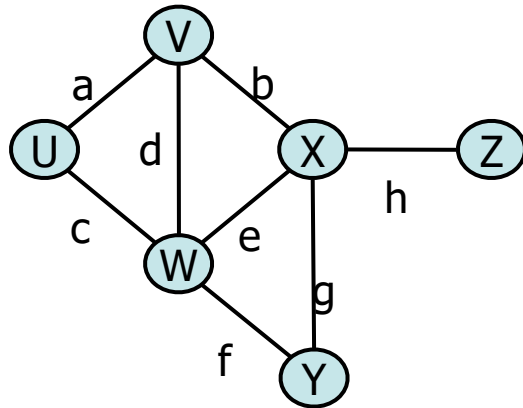
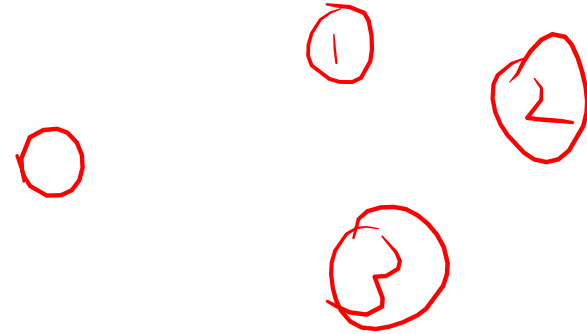
Acyclic subgraph( $G$ )

Spanning tree( $G$ )

Running times are often reported by **n**, the number of vertices, but often depend on **m**, the number of edges.

How many edges? **Minimum edges:**

Not Connected:



Connected\*:

**Maximum edges:**

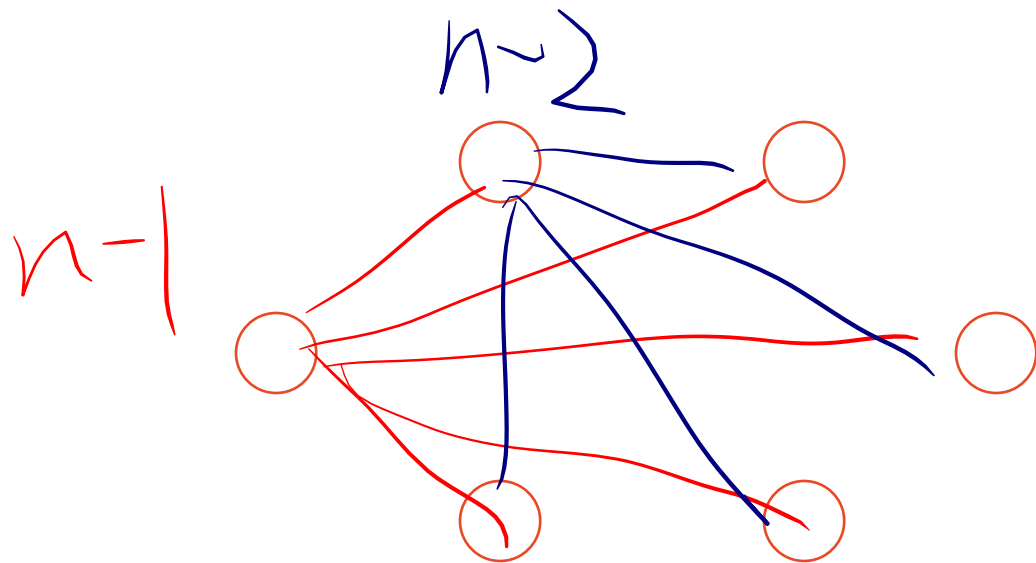
Simple:

Not simple:

$$\sum_{v \in V} \deg(v) =$$

$n-1$   
 $\frac{n(n-1)}{2}$   
 $+\infty \rightarrow$  no upper bound on non-simple

# Max edges in a simple graph



$$\binom{n}{2}$$

$$(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$$

Running times are often reported by **n**, the number of vertices, but often depend on **m**, the number of edges.

How many edges? **Minimum edges:**

Not Connected:

① ② ③

Connected\*:

$n-1$

**Maximum edges:**

Simple:

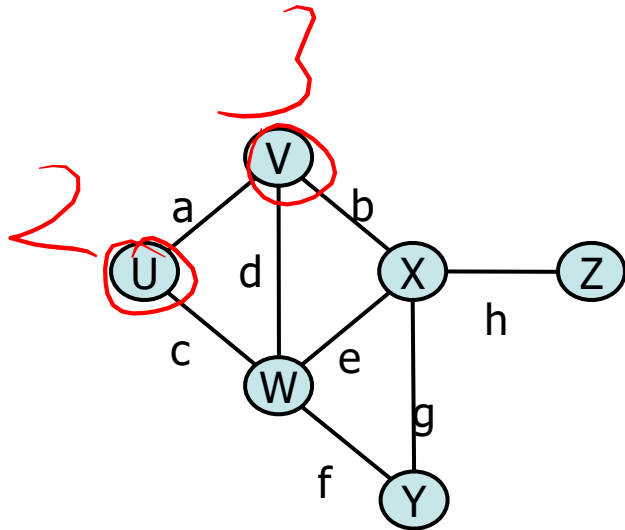
$\frac{n(n-1)}{2}$

Not simple:

$+\infty \rightarrow$  no upper

$$\sum_{v \in V} \deg(v) = 2m$$

simple bound on non-simple



# Proving the size of a minimally connected graph

## Theorem:

Every minimally connected graph  $G=(V, E)$  has  $|V|-1$  edges.

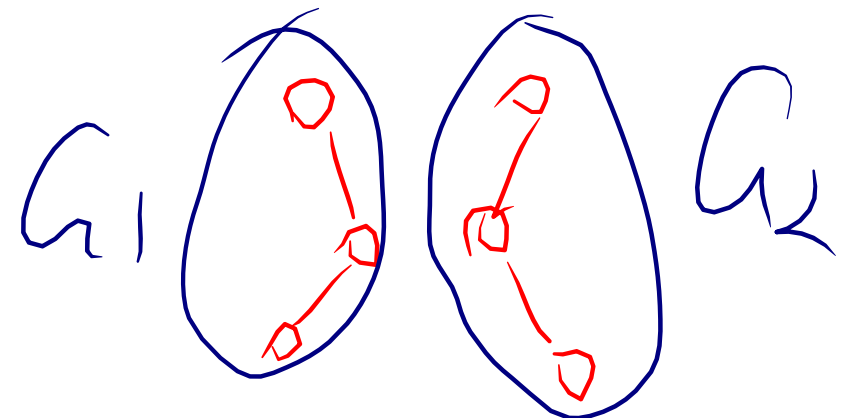
$n-1$

**Thm:** Every minimally connected graph  $G=(V, E)$  has  $|V|-1$  edges.

**Proof:** Consider an arbitrary, minimally connected graph  $G=(V, E)$ .

**Lemma 1:** Every connected subgraph of  $G$  is minimally connected.  
(*Easy proof by contradiction left for you.*)

**Inductive Hypothesis:** For any  $j < |V|$ , any minimally connected graph of  $j$  vertices has  $j-1$  edges.



**Suppose  $|V| = 1$ :**

**Definition:** A minimally connected graph of 1 vertex has 0 edges.

**Theorem:**  $|V| - 1$  edges  $\rightarrow 1 - 1 = 0$ .



$$\begin{array}{l} \text{Graph of } |V| = 1 \\ \hline |E| = 0 \end{array}$$

Suppose  $|V| > 1$ :

Choose any vertex  $u$  and let  $d$  denote the degree of  $u$ .

Remove the incident edges of  $u$ , partitioning the graph into \_\_\_\_\_ components:  $C_0 = (V_0, E_0), \dots, C_d = (V_d, E_d)$ .

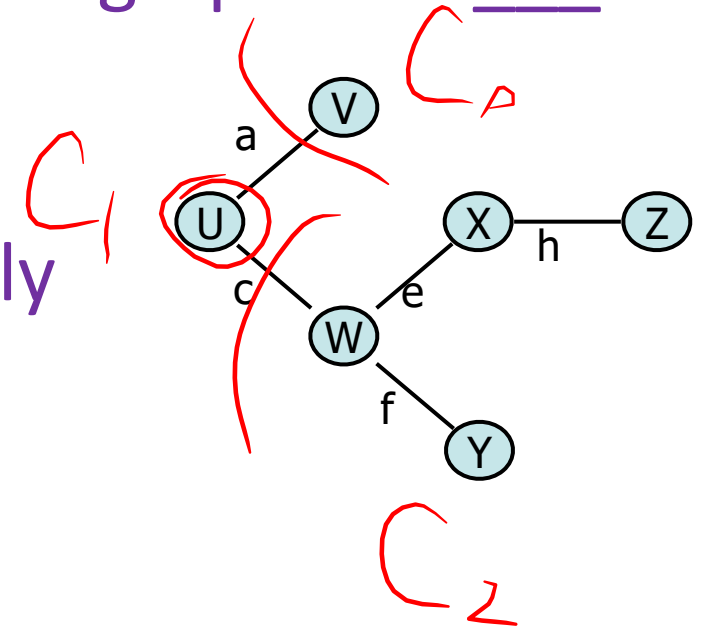
By Lemma 1, every component  $C_k$  is a minimally connected subgraph of  $G$ .

By our IH:  $|E_k| = |V_k| - 1$ .

Finally, we count edges:

$$= d + \sum_{k=1}^d |E_k| = d + \sum_{k=1}^d (|V_k| - 1) = \sum_{k=1}^d |V_k| = n - 1$$

$d=2$



# CS 225

## Data Structures

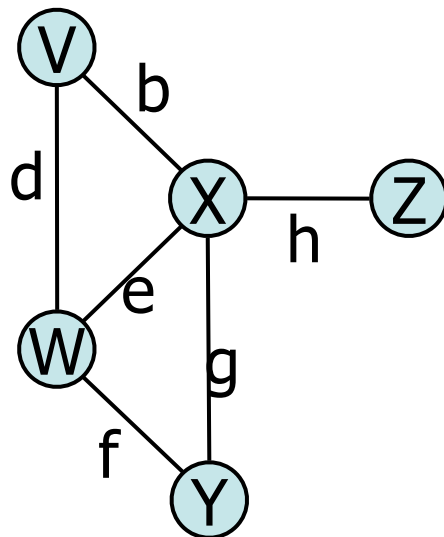
**Volodymyr Kindratenko**

*Slides courtesy of Wade Fagen-Ulmschneider*

# Graph ADT

## Data:

- Vertices
- Edges
- Some data structure maintaining the structure between vertices and edges.



## Functions:

- insertVertex(K key);
- insertEdge(Vertex v1, Vertex v2, K key);
- removeVertex(Vertex v);
- removeEdge(Vertex v1, Vertex v2);

- incidentEdges(Vertex v); *→ iterator*

- areAdjacent(Vertex v1, Vertex v2);

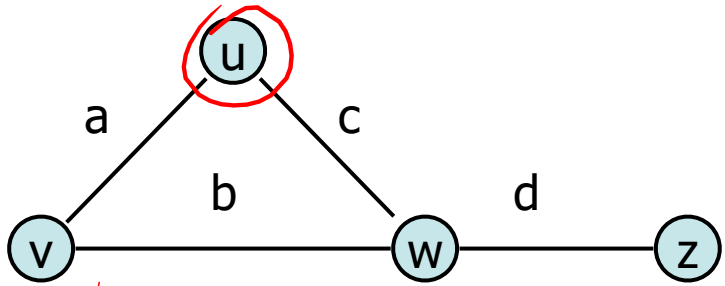
*Directed graph only*

- origin(Edge e);

- destination(Edge e);

*$(u, v) \neq (v, u)$*

# Graph Implementation: Edge List



vertex

edges

u  
v  
w  
z

~~u v a~~  
v w b  
~~u w c~~  
w z d

`insertVertex(K key);`

$O(1)$

`removeVertex(Vertex v);`

$O(m)$

`areAdjacent(Vertex v1, Vertex v2);`

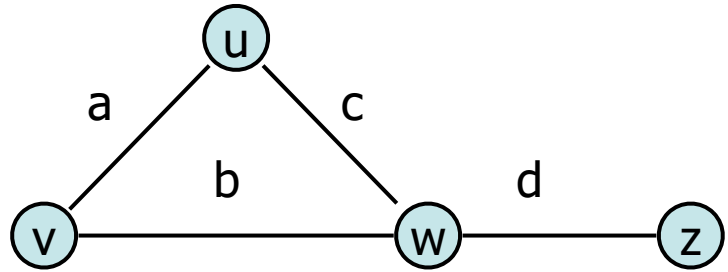
$O(m)$

`incidentEdges(Vertex v);`

$O(m)$

$O(1)$

# Graph Implementation: Adjacency Matrix



insertVertex(K key) :

$O(n)^2$

removeVertex(Vertex v) :

areAdjacent(Vertex v1, Vertex v2) :

incidentEdges(Vertex v) :

$O(n^2)$   
n

|   |
|---|
| u |
| v |
| w |
| z |

+

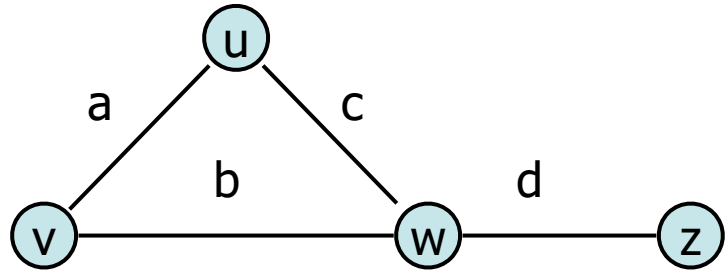
|   |   |   |
|---|---|---|
| u | v | a |
| v | w | b |
| u | w | c |
| w | z | d |

|   | u | v | w | z |
|---|---|---|---|---|
| u | 0 | 1 | 1 | 0 |
| v | 1 | 0 | 1 | 0 |
| w | 1 | 1 | 0 | 1 |
| z | 0 | 0 | 1 | 0 |

+

double size

# Graph Implementation: Adjacency Matrix



insertVertex(K key) :

$O(n)^2$

removeVertex(Vertex v) :

areAdjacent(Vertex v1, Vertex v2) :

incidentEdges(Vertex v) :

$O(n^2)$   
n

|   |
|---|
| u |
| v |
| w |
| z |

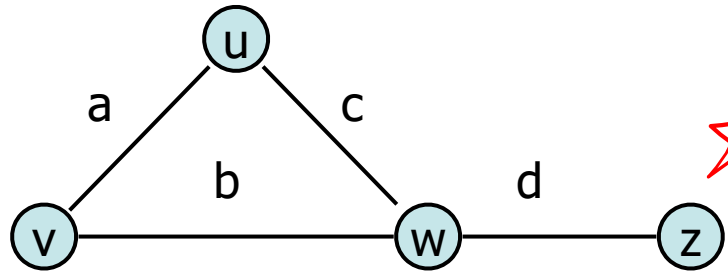
+

|   |   |   |
|---|---|---|
| u | v | a |
| v | w | b |
| u | w | c |
| w | z | d |

|   | u | v | w | z |
|---|---|---|---|---|
| u | 0 | 1 | 1 | 0 |
| v | 1 | 0 | 1 | 0 |
| w | 1 | 1 | 0 | 1 |
| z | 0 | 0 | 1 | 0 |

double size

# Graph Implementation: Adjacency Matrix



insertVertex(K key) :

removeVertex(Vertex v) :

★ areAdjacent(Vertex v1, Vertex v2) :

incidentEdges(Vertex v) :

$O(n)^2$

$O(n)$

$O(1)$

$\frac{O(n^2)}{n}$

new edge?

|   |
|---|
| u |
| v |
| w |
| z |

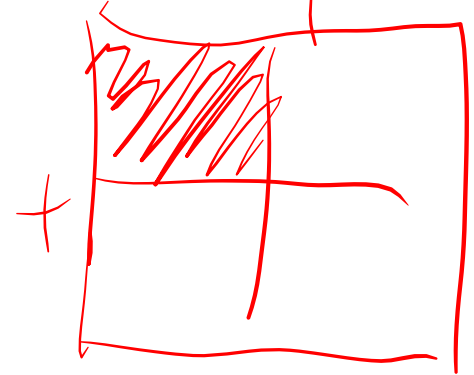
[+]

|              |              |              |
|--------------|--------------|--------------|
| <del>u</del> | <del>v</del> | <del>a</del> |
| v            | w            | b            |
| <del>u</del> | <del>w</del> | <del>c</del> |
| w            | z            | d            |

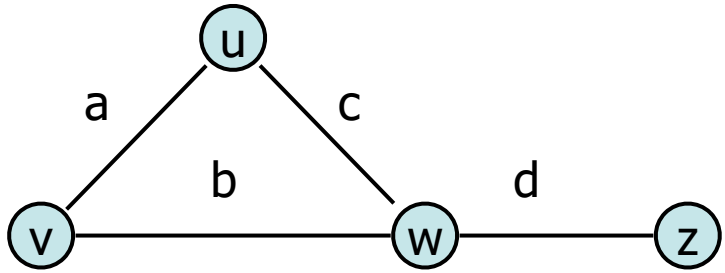
[u|z|e]

|   | u | v | w | z |
|---|---|---|---|---|
| u |   |   |   |   |
| v |   |   |   |   |
| w |   |   |   |   |
| z |   |   |   |   |

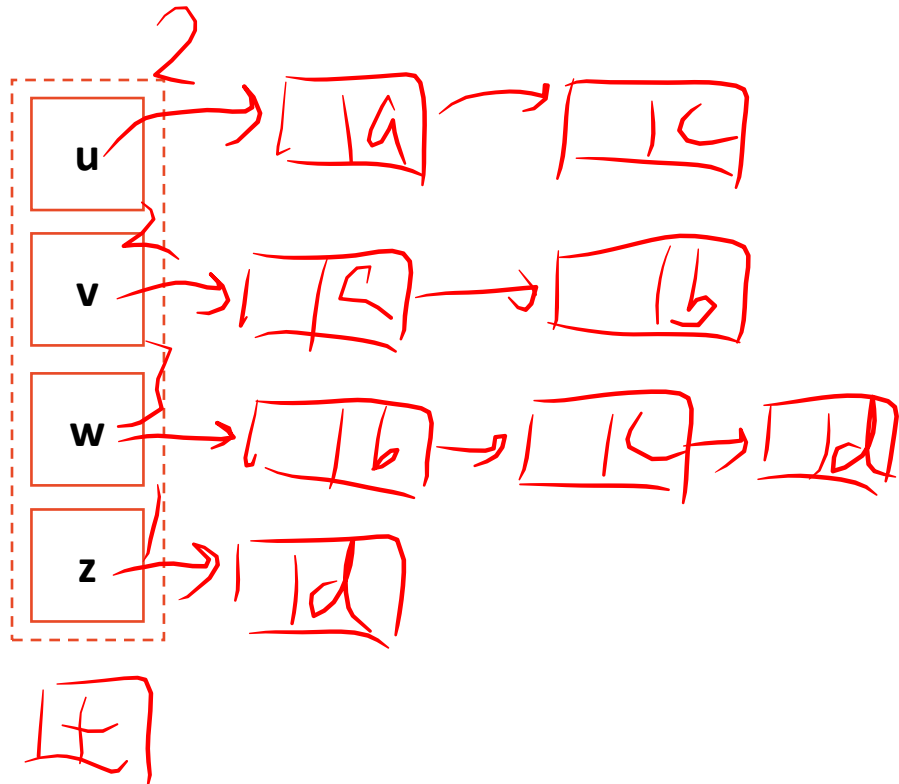
double size



# Graph Implementation: Adjacency List

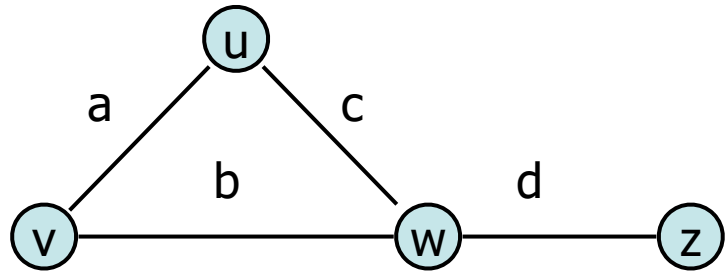


**insertVertex(K key);**  
**removeVertex(Vertex v);**  
**areAdjacent(Vertex v1, Vertex v2);**  
**incidentEdges(Vertex v);**



|   |   |   |
|---|---|---|
| u | v | a |
| v | w | b |
| u | w | c |
| w | z | d |

# Graph Implementation: Adjacency List



**insertVertex(K key);**

**removeVertex(Vertex v);**

**areAdjacent(Vertex v1, Vertex v2);**

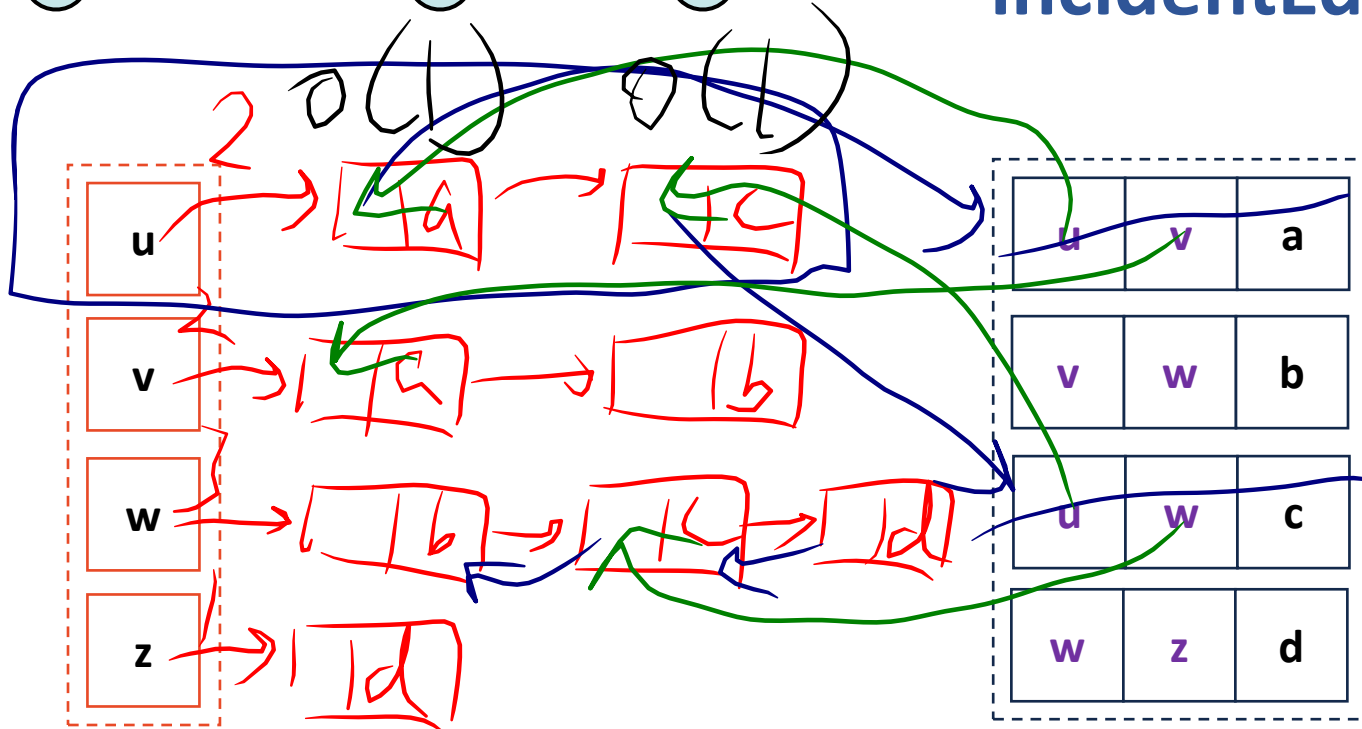
**incidentEdges(Vertex v);**

$O(1)$

$O(\deg(v))$

$O(\deg(v))$

$O(\min(\deg(v_1), \deg(v_2)))$



[+]



# CS 225

## Data Structures

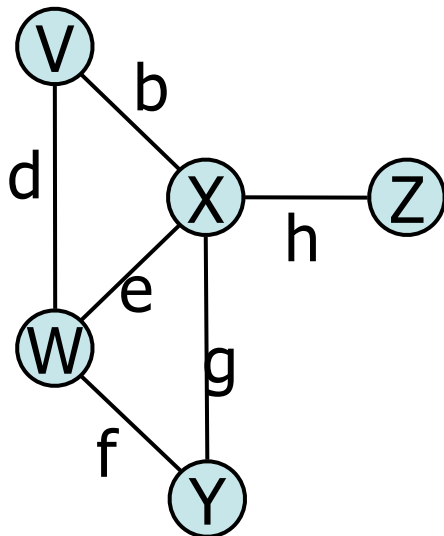
**Volodymyr Kindratenko**

*Slides courtesy of Wade Fagen-Ulmschneider*

# Graph ADT

## Data:

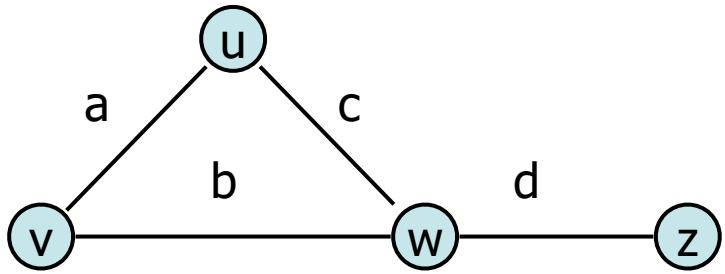
- Vertices
- Edges
- Some data structure maintaining the structure between vertices and edges.



## Functions:

- insertVertex(K key);
- insertEdge(Vertex v1, Vertex v2, K key);
- removeVertex(Vertex v);
- removeEdge(Vertex v1, Vertex v2);
- incidentEdges(Vertex v);
- areAdjacent(Vertex v1, Vertex v2);
- origin(Edge e);
- destination(Edge e);

# Edge List



## Key Ideas:

- Given a vertex,  $O(1)$  lookup in vertex list
  - Implement w/ a hash table, etc.
- All basic ADT operations runs in  $O(m)$  time

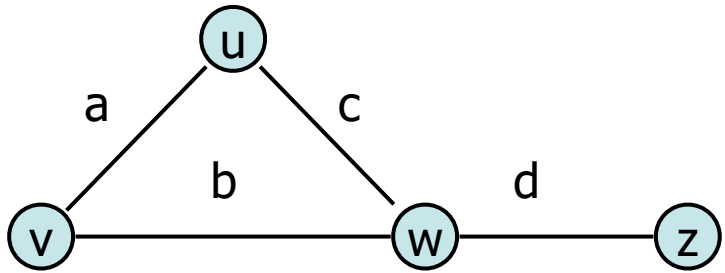
### Vertex List

|   |
|---|
| u |
| v |
| w |
| z |

### Edge List

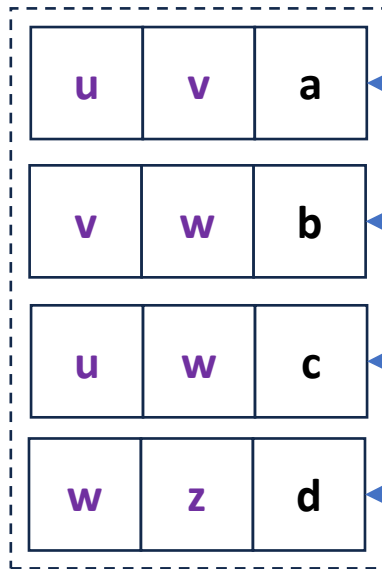
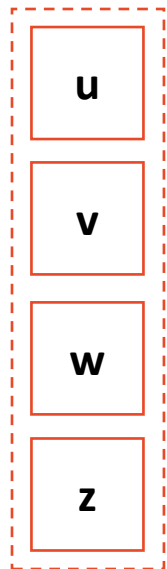
|   |   |   |
|---|---|---|
| u | v | a |
| v | w | b |
| u | w | c |
| w | z | d |

# Adjacency Matrix



## Key Ideas:

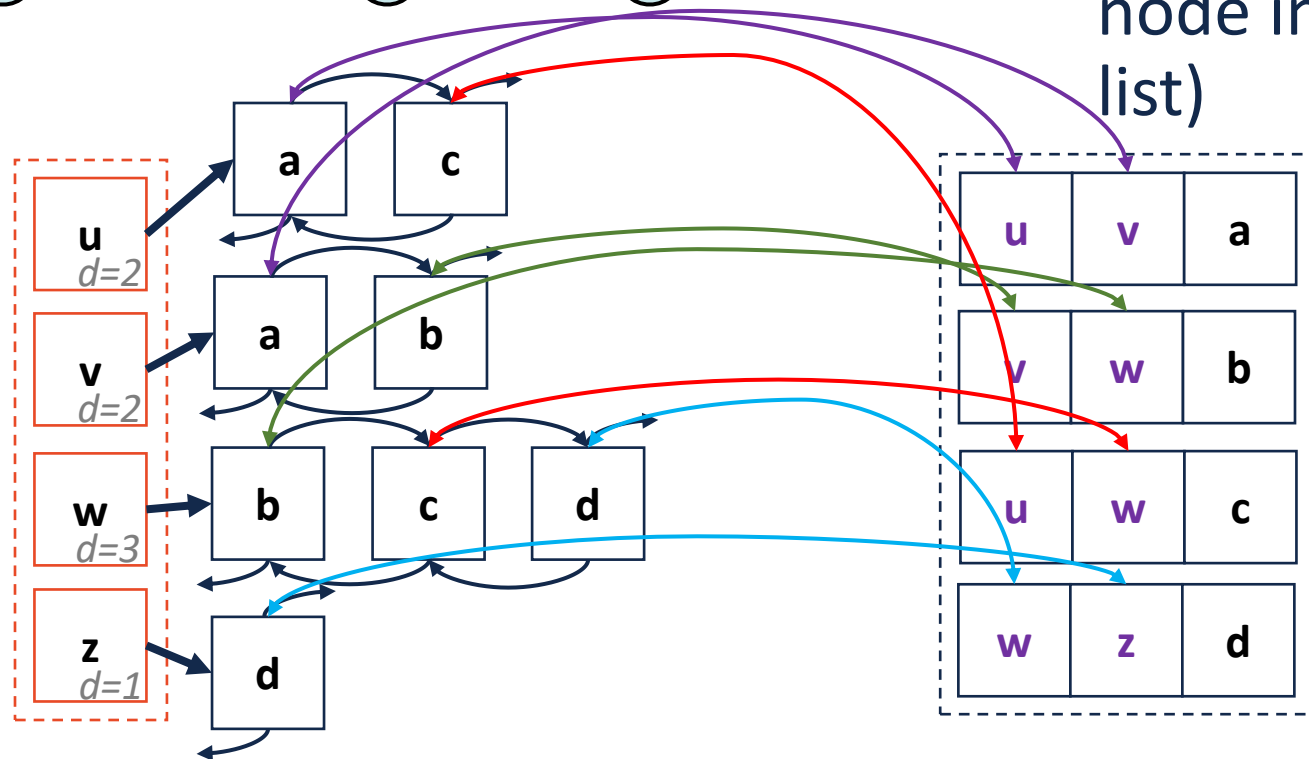
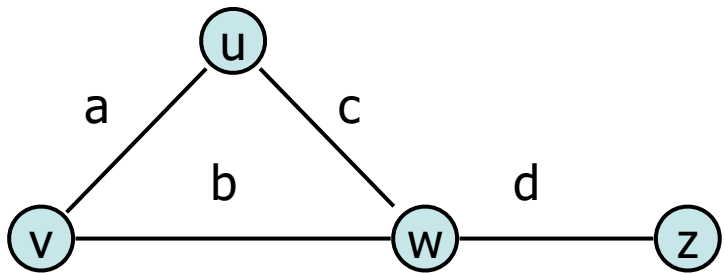
- Given a vertex,  $O(1)$  lookup in vertex list
- Given a pair of vertices (an edge),  $O(1)$  lookup in the matrix
- Undirected graphs can use an upper triangular matrix



|   | u | v | w | z |
|---|---|---|---|---|
| u |   | ∅ | ● | ∅ |
| v |   |   | ∅ | ∅ |
| w |   |   |   | ∅ |
| z |   |   |   |   |

$n^2$  space

# Adjacency List



## Key Ideas:

- $O(1)$  lookup in vertex list
- Vertex list contains a doubly-linked adjacency list
  - $O(1)$  access to the adjacent vertex's node in adjacency list (via the edge list)
- Vertex list maintains a count of incident edges, or **deg(v)**
- Many operations run in  $O(\deg(v))$ , and  $\deg(v) \leq n-1$ ,  $O(n)$ .

$$n^2 + m \rightarrow o(n^2)$$

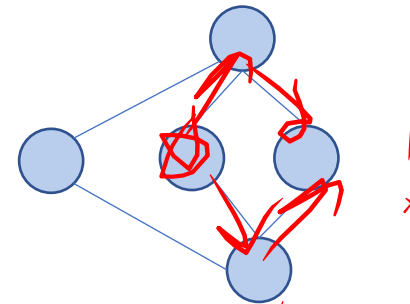
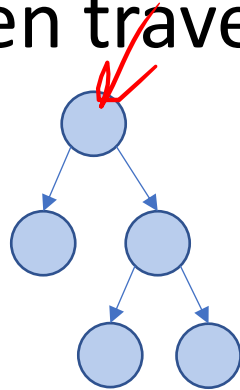
| Expressed as big-O  | Edge List              | Adjacency Matrix       | Adjacency List                       |
|---------------------|------------------------|------------------------|--------------------------------------|
| Space               | $n+m$                  | $n^2$                  | $n+m$                                |
| insertVertex(v)     | 1 ✱                    | n                      | 1 ✱                                  |
| removeVertex(v)     | $m \geq \text{deg}(v)$ | $n \geq \text{deg}(v)$ | $\text{deg}(v)$ ✱                    |
| insertEdge(v, w, k) | 1 ✱                    | 1 ✱                    | 1 ✱                                  |
| removeEdge(v, w)    | 1 ✱                    | 1 ✱                    | 1 ✱                                  |
| incidentEdges(v)    | m                      | n                      | $\text{deg}(v)$ ✱                    |
| areAdjacent(v, w)   | m                      | 1 ✱                    | $\min(\text{deg}(v), \text{deg}(w))$ |

# Traversal:

**Objective:** Visit every vertex and every edge in the graph.

**Purpose:** Search for interesting sub-structures in the graph.

We've seen traversal before ....but it's different:



In-order

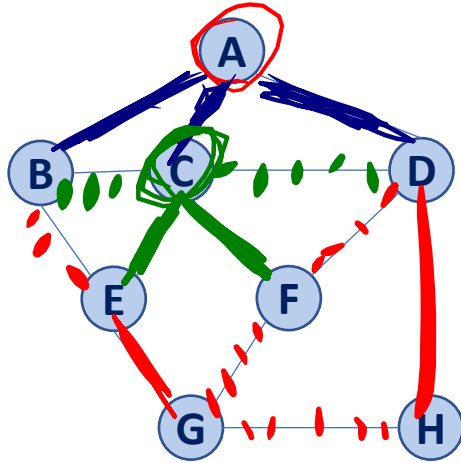
Post-order

Level-order

- Ordered
- Obvious Start
- Notion of doneeness

- No clear order
- Arbitrary start
- Unclear notion of doneeness

# Traversal: BFS



Discovery edge

cross edge  
- - - -



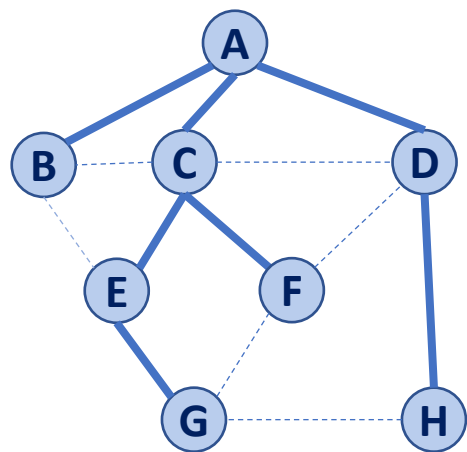
|              |   |   |                  |
|--------------|---|---|------------------|
| <del>0</del> | - | A | <del>CBD</del>   |
| 1            | A | B | ACE              |
| 1            | A | C | <del>ABDEF</del> |
| 1            | A | D | ACFH             |
|              |   | E | BCG              |
|              |   | F | CDG              |
|              |   | G | EFH              |
|              |   | H | DG               |

Adjacency list

~~G~~ ~~A~~ ~~F~~ ~~E~~ ~~D~~ ~~B~~ ~~C~~ ~~A~~

Queue

# Traversal: BFS



| d | p |   | Adjacent Edges |
|---|---|---|----------------|
| 0 | A | A | C B D          |
| 1 | A | B | A C E          |
| 1 | A | C | B A D E F      |
| 1 | A | D | A C F H        |
| 2 | C | E | B C G          |
| 2 | C | F | C D G          |
| 3 | E | G | E F H          |
| 2 | D | H | D G            |

